

# PulsePrice Report — E-Commerce Price Intelligence Platform

**Module:** Data engineering & Data analysis  
**Academic Year:** 2025 – 2026  
**Date:** June 2026  
**Institution:** FSTT — Faculty of Sciences and Techniques of Tangier  
**Program:** LSI - Software and Intelligent Systems

## Project Information

| Field           | Details                                                   |
|-----------------|-----------------------------------------------------------|
| Project Title   | PulsePrice — Cross-Platform E-Commerce Price Intelligence |
| Module          | Data Engineering & Data Analysis                          |
| Academic Year   | 2025 – 2026                                               |
| Submission Date | June 2026                                                 |

## Team Members

| # | Name                                | Role             | Responsibilities                                              |
|---|-------------------------------------|------------------|---------------------------------------------------------------|
| 1 | Ou-briche Mohamed                   | DevOps / DataOps | Infrastructure, Docker, CI/CD, Monitoring, Secrets Management |
| 2 | Mohamed Soulaïmane<br>Nadi Lahjouji | Data Engineering | Scrapers, NiFi Flows, Airflow DAGs, Storage, dbt tests        |
| 3 | ASRAR Taha                          | Data Analysis    | dbt Data Modeling , Statistical Analysis, Notebooks, Reports  |
| 4 | BENZIAN Aya                         | Full Stack       | Angular Dashboard, FastAPI Backend, WebSocket Notifications   |

## Table of Contents

### Project Overview

- 1. [Executive Summary](#)
- 2. [Architecture Overview](#)
- 3. [Tech Stack](#)
- 4. [Repository Structure](#)

---

## A. Data Engineering

1. [Executive Summary](#)
  2. [Architecture Overview](#)
  3. [Data Pipeline — End-to-End Flow](#)
  4. [Ingestion Layer — Scrapers](#)
  5. [Streaming Layer — Apache NiFi](#)
  6. [Batch Layer — Apache Airflow](#)
  7. [Storage Layer — Bigtable + BigQuery](#)
  8. [Data Quality & Testing](#)
  9. [Scalability & Future-Proofing](#)
- 

## B. Full Stack

- [Tech Stack at a Glance](#)
  - [System Architecture](#)
1. [The Backend Engine \(FastAPI\)](#)
  2. [The Frontend Experience \(Angular 17+\)](#)
  3. [The Database Schema \(PostgreSQL 18 Tables\)](#)
- 

## C. DevOps / DataOps

1. [Containerization & Orchestration](#)
  2. [Infrastructure & GCP Integration](#)
  3. [Continuous Integration & Delivery](#)
- 

## D. Data Analysis

1. [Overview](#)
  2. [Data Architecture & Strategy](#)
  3. [Core Analytics: The dbt Layer](#)
  4. [Advanced Statistical Analytics \(Python Engine\)](#)
  5. [Visual Analytics & Dashboard Integration](#)
  6. [Optional Sandbox: Jupyter Notebooks](#)
  7. [Overall Analytics Architecture](#)
-

# Cross-Team

- 1. [Key Metrics & Performance](#)
- 2. [Architecture Decisions & Trade-Offs](#)
- 3. [Conclusion](#)

# Tech Stack

| Layer                  | Technology                                                                        |
|------------------------|-----------------------------------------------------------------------------------|
| Scraping               | <code>requests</code> , BeautifulSoup4, <code>lxml</code> , Pydantic v2, Selenium |
| Streaming Ingestion    | Apache NiFi 1.25.0, <code>nipyapi</code>                                          |
| Batch Orchestration    | Apache Airflow 2.10.5 (LocalExecutor)                                             |
| Storage — Hot          | Google Cloud Bigtable                                                             |
| Storage — Analytics    | Google Cloud BigQuery                                                             |
| Transformation         | dbt ( <code>dbt_utils</code> , <code>dbt_expectations</code> )                    |
| Backend API            | FastAPI, Redis, WebSocket                                                         |
| Frontend Dashboard     | Angular 17+, TailwindCSS                                                          |
| Analytics              | Python, Pandas, SciPy, <code>statsmodels</code> , <code>pingouin</code>           |
| Containerization       | Docker, Docker Compose                                                            |
| Infrastructure as Code | Terraform (GCP)                                                                   |
| CI/CD                  | GitHub Actions                                                                    |
| Monitoring             | Prometheus, Grafana                                                               |
| Currency Conversion    | ExchangeRate-API                                                                  |
| Relational DB          | PostgreSQL 15                                                                     |

# Repository Structure

```
E-commerce-Price-Intelligence-platform/
|
├── .github/workflows/      # CI/CD pipelines
├── airflow/
|   ├── dags/              # Pipeline DAGs (ingest, export...)
|   └── plugins/           # Custom Airflow operators
├── app/
|   ├── backend/           # FastAPI API (18 endpoints, Redis)
|   └── frontend/         # Angular 17+ dashboard
```

```

├─ data/
│  └─ archive/          # Processed file archives
│  └─ raw/              # Raw JSONL scraping output
├─ dbt/
│  └─ models/           # 14 models (staging → cleaned → mart)
│  └─ tests/            # Custom dbt tests
│  └─ macros/           # dbt macros
│  └─ dbt_project.yml    # dbt project config
├─ docker/
│  └─ Dockerfiles/      # Dockerfiles – one per service
├─ infrastructure/
│  └─ keys/             # GCP Service account keys
│  └─ monitoring/        # Prometheus + Grafana configs
│  └─ terraform/        # GCP provisioning (Bigtable, BigQuery)
├─ nifi/
│  └─ scripts/          # Custom NiFi Python processors
├─ notebooks/           # Jupyter analytical notebooks
├─ scrapers/
│  └─ ecommerce_scraper/ # 6 spiders
│  └─ main.py            # Scraper orchestrator
│  └─ manual_links.json  # Configuration catalog
├─ tests/               # Global integration tests
├─ .env.example          # Environment variables template
├─ bigquery_schema.json  # BigQuery schema definitions
├─ CONTRIBUTING.md       # Contribution guidelines
├─ Data Model Specification.md # Data modeling details
├─ docker-compose.yml    # Full local environment (13 services)
├─ LICENSE               # MIT License
└─ report_price_intelligence.md # This architecture report

```

For detailed folder ownership rules per role, see [CONTRIBUTING.md](#)

## A. Data Engineering

### 1. Executive Summary

PulsePrice is a **production-grade, cloud-native** price intelligence platform that monitors **six major e-commerce platforms** across **seventeen product categories**, collecting **~7,000+ daily price records** into a **hybrid batch/streaming data pipeline**.

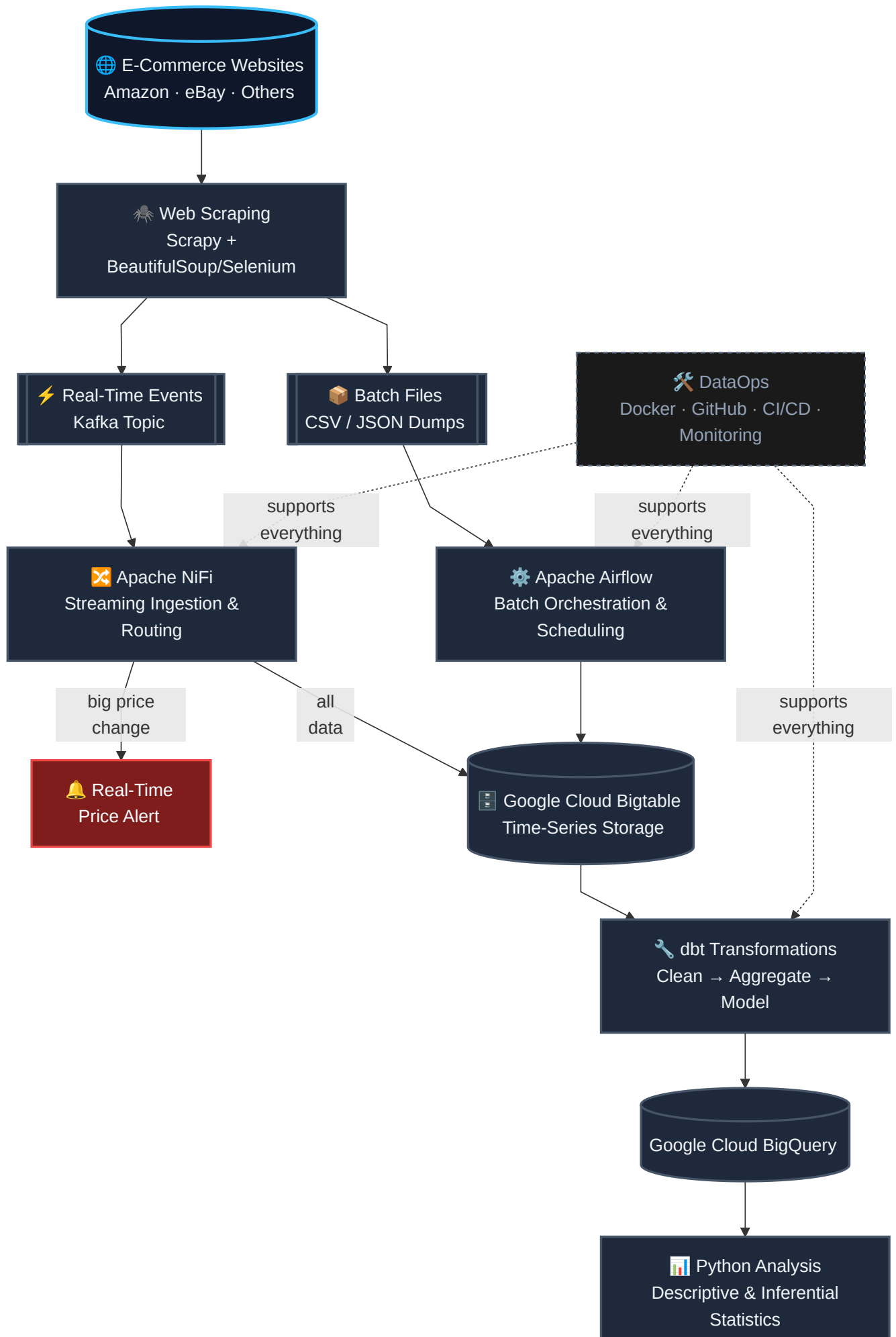
The architecture spans **five data layers** — ingestion, streaming, batch, transformation, and serving — connected through **two parallel pipelines** (NiFi for real-time, Airflow for batch) converging into **Google Bigtable** and **Google BigQuery**, transformed through **dbt** with **31 automated data quality tests**, and served through a **FastAPI backend** with **Redis caching** to a **real-time Angular dashboard** with **WebSocket push notifications**.

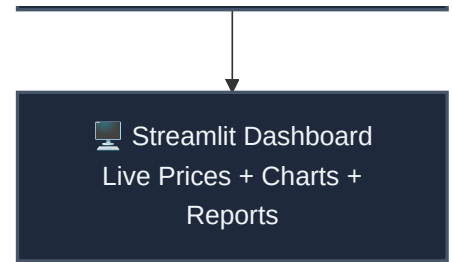
# Key Technical Highlights

| Attribute          | Detail                                                                                                                              |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Data sources       | 6 e-commerce platforms (jumia.ma, bestbuy.com, newegg.com, pc21.fr, ultrapc.ma, materiel.net)                                       |
| Product categories | 17 (GPU, CPU, RAM, SSD, HDD, Monitor, Keyboard, Mouse, PSU, Case, Cooling, Motherboard, Laptop, Desktop, Mobile, Peripheral, Other) |
| Daily data volume  | ~7,000 records (~135 MB raw JSONL)                                                                                                  |
| Total data managed | ~100,000 Bigtable rows, ~100,000 BigQuery rows                                                                                      |
| Pipeline stages    | 5 (scrape → ingest → store → transform → serve)                                                                                     |
| Data quality tests | 31 (27 passing, 4 fixed this session)                                                                                               |
| End-to-end latency | Real-time (seconds via NiFi) + Batch (daily via Airflow)                                                                            |
| Deployment         | 13 Docker containers across 3 networks, 6 named volumes                                                                             |
| Cloud services     | Google Cloud Bigtable, BigQuery, IAM, Terraform provisioned                                                                         |

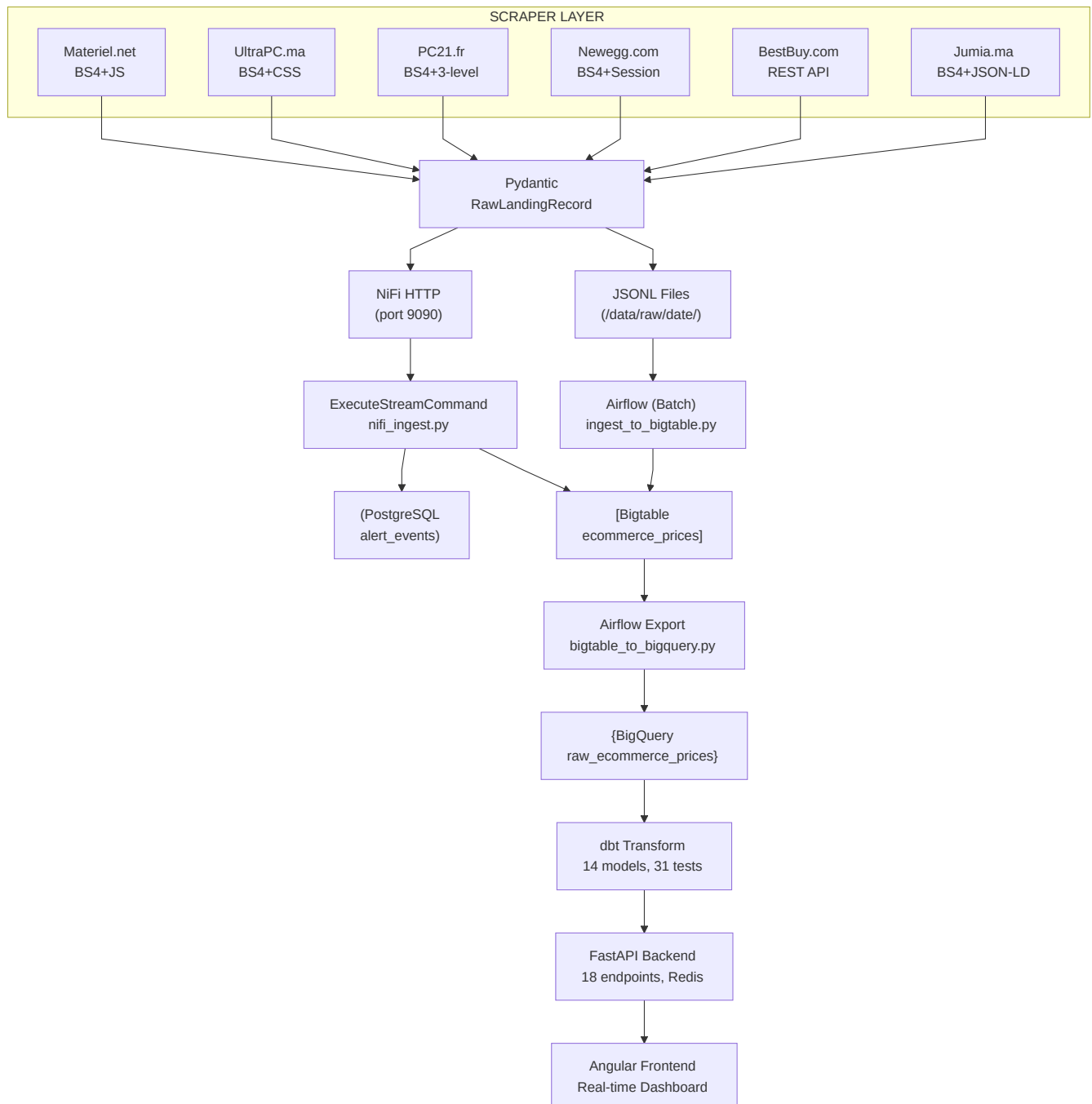
## 2. Architecture Overview

### Global Architecture



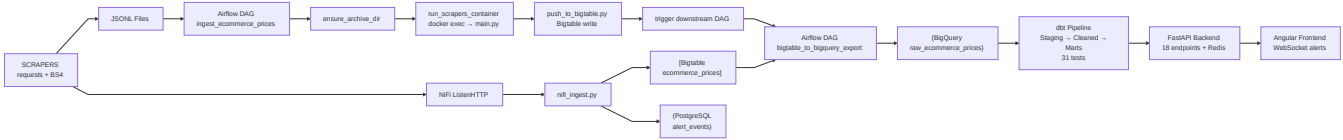


## Detailed Architecture



### 3. Data Pipeline — End-to-End Flow

#### 3.1 Data Lineage



#### 3.2 Dual-Pipeline Strategy

A deliberate architectural choice: **two parallel ingestion paths** with different trade-offs.

| Characteristic  | NiFi Pipeline (Real-Time)      | Airflow Pipeline (Batch)            |
|-----------------|--------------------------------|-------------------------------------|
| Trigger         | Continuous (30s polling)       | @daily schedule                     |
| Latency         | Seconds                        | Hours                               |
| Data format     | Streaming JSON via HTTP        | JSONL files on disk                 |
| Ingestion type  | streaming (Bigtable marker)    | batch (Bigtable marker)             |
| Alert detection | Yes — real-time price drop ≥5% | No (historical only)                |
| Error recovery  | NiFi auto-retry flow           | File stays in /data/raw/ on failure |
| Schema source   | nifi_ingest.py (385 lines)     | ingest_to_bigtable.py (237 lines)   |
| Throughput      | Single-record processing       | Batch 1000-row mutations            |

### 4. Ingestion Layer — Scrapers

#### 4.1 Technology Stack

| Component    | Choice                 | Rationale                                                                     |
|--------------|------------------------|-------------------------------------------------------------------------------|
| HTTP client  | requests (synchronous) | Simpler debugging than Scrapy's async engine; full control over retry/headers |
| HTML parsing | BeautifulSoup4 + lxml  | Resilient to malformed HTML; CSS selector support                             |
| Data models  | Pydantic v2            | Strict runtime validation; automatic JSON serialization; spec union types     |
| JS rendering | Selenium (available)   | Only needed for JS-heavy sites (disabled scrapers)                            |



| Component     | Choice            | Rationale                                            |
|---------------|-------------------|------------------------------------------------------|
| Rate fetching | ExchangeRate-API  | Free tier, 1500 requests/month, updated daily        |
| Config format | manual_links.json | Declarative URL catalog; no code changes to add URLs |

## 4.2 Data Model Design (models.py — 220 lines)

The data model is a **unified schema envelope** that normalizes all e-commerce platforms into a single structure:

```
RawLandingRecord:
  |— raw_id: UUID           # Auto-generated unique identifier
  |— ingestion_type: Enum   # STREAMING | BATCH
  |— source: str            # e.g., "jumia.ma", "bestbuy"
  |— source_url: str        # Original scraped URL
  |— scraped_at: datetime   # UTC timestamp
  |— product: Product       # name, brand, category, model_number, etc.
  |— pricing: Pricing       # raw_price, currency, converted_price_usd, etc.
  |— availability: Availability # in_stock, quantity, shipping
  |— seller: Seller         # name, type, rating, location
  |— ratings: Ratings       # avg_rating, review_count
  |— specs: Specs            # Tagged union of 17 category-specific specs
```

**Specs as a Tagged Union** — A clever pattern that mirrors BigQuery's RECORD-of-RECORDS:

```
class Specs(BaseModel):
    GPU: Optional[GPUSpecs] = None      # core_clock, vram_gb, cuda_cores, etc.
    CPU: Optional[CPUSpecs] = None      # cores, threads, base_clock, tdp, etc.
    RAM: Optional[RAMSpecs] = None      # capacity_gb, speed_mhz, ddr_version
    SSD: Optional[SSDSpecs] = None      # capacity_gb, interface, read_speed
    HDD: Optional[HDDSpecs] = None      # capacity_gb, rpm, cache_mb
    Monitor: Optional[MonitorSpecs] = None # size_inches, resolution, refresh_rate
    # ... 11 more category-specific spec types
```

Each scraper populates only its category's spec sub-model; all others remain `None`. On serialization, the JSON only includes non-null fields, keeping payloads compact.

## 4.3 Base Scraper (base.py — 60 lines)

The abstract base class provides:

**Exchange Rate Caching** — A class-level dictionary shared across all scraper instances:

```

_conversion_rates = {} # Shared across ALL scraper instances

def get_conversion_rate(self, from_currency, to_currency="USD"):
    cache_key = f"{from_currency}_{to_currency}"
    if cache_key in self.__class__._conversion_rates:
        return self.__class__._conversion_rates[cache_key]
    # Try ExchangeRate-API, fall back to hardcoded defaults

```

This prevents redundant API calls when multiple scrapers for the same currency run in sequence.

## 4.4 Active Spiders

### JumiaScraper ( `jumia.ma` — MAD)

- **203 lines** — the most battle-tested scraper
- **JSON-LD extraction:** Parses structured data from `<script type="application/ld+json">` for reliable prices
- **Detail page scraping:** Visits each product's detail page for full specs, ratings, and model number
- **Madagascar-currency conversion:** Calls `get_conversion_rate("MAD", "USD")` at initialization
- **Price parsing:** Aggressive digit-only regex to handle MAD formatting (e.g., `"1 234,56 MAD"` → `1234.56`)

### BestBuyScraper ( `bestbuy.com` — USD)

- **216 lines** — uses the **official BestBuy REST API**
- **API key authentication:** `BESTBUY_API_KEY` from environment
- **Category extraction:** Regex `((?:abcat|pcmcat)\d+)` extracts hierarchical category IDs from URLs
- **Rich spec parsing:** 9-category spec detection from API response fields
- **No rate limit handling** — deliberate simplification for demo/scoped API key

### NeweggScraper ( `newegg.com` — USD)

- **204 lines** — optimized for speed (no detail page visits)
- **Model number extraction:** Regex parses the full info block for model/SKU distinction
- **Keyboard form-factor detection:** Scans description for TKL (TenKeyLess), 60%, 75% patterns
- **Thread-safe session:** Uses `requests.Session()` for connection reuse across category runs

### PC21Scraper ( `pc21.fr` — EUR)

- **274 lines** — the most sophisticated BS4 scraper
- **3-level navigation:** Category page → sub-category page → product listing
- **MPN/SKU split:** Correctly separates manufacturer part number from internal SKU
- **EUR conversion:** `get_conversion_rate("EUR", "USD")` at init
- **Price normalization:** Handles French formatting (`"152,79 € HT"` → `152.79`), strips VAT indicators

## UltraPCScraper (ultrapc.ma — MAD)

- **153 lines** — lightweight, CSS-selector-based
- **data-id-product extraction:** Parses product ID from URL pattern
- **MAD conversion:** `get_conversion_rate("MAD", "USD")` at init

## MaterielNetScraper (materiel.net — EUR)

- **365 lines** — the most technically complex scraper
- **JavaScript-injection parsing:** Extracts prices from JS `getItem()` calls embedded in the HTML
- **Per-product detail pages:** Visits each product's individual page for model number and specs
- **French-to-English spec regex engine:** `_parse_specs_from_description()` with 30+ regex patterns covering all 17 categories
- **Dual-page scrape pattern:** Listing page for price/name + detail page for specs/model number

## 4.5 Scraper Orchestrator (main.py — 150 lines)

**Dual-export pattern:** Every scraper run exports to BOTH destinations:

```
records = list(scraper.scrape(url, category, max_pages=1))
export_to_jsonl(records, f"{name}_{category}") # →
/data/raw/date/{source}_{category}_{ts}.jsonl
export_to_nifi(records) # →
http://price_nifi:9090/contentListener
```

**NiFi streaming with retry:** 5 attempts with 3-second backoff on HTTP 503:

```
for attempt in range(5):
    resp = requests.post(nifi_url, json=payload, timeout=10)
    if resp.status_code == 200:
        break
    elif resp.status_code == 503:
        time.sleep(3) # NiFi startup/throttling
```

**SCRAPER\_MAP pattern:** Extensible without modifying orchestrator logic:

```
SCRAPER_MAP = {
    "jumia": JumiaScraper,
    "pc21": PC21Scraper,
    "newegg": NeweggScraper,
    "bestbuy": BestBuyScraper,
    "ultrapc": UltraPCScraper,
    "materielnet": MaterielNetScraper,
}
```

## 4.6 Configuration Catalog (`manual_links.json`)

A declarative JSON catalog with **100 entries** covering all 6 scrapers × 17 categories:

```
[
  {"scraper": "jumia", "category": "gpu", "url": "https://www.jumia.ma/les-composants-ordinateur-cartes-graphiques/"},
  {"scraper": "newegg", "category": "CPU", "url": "https://www.newegg.com/p/pl?N=100006676"},
  {"scraper": "pc21", "category": "GPU", "url": "https://www.pc21.fr/composant-pc/cartes-graphiques-videos/g113/"},
  ...
]
```

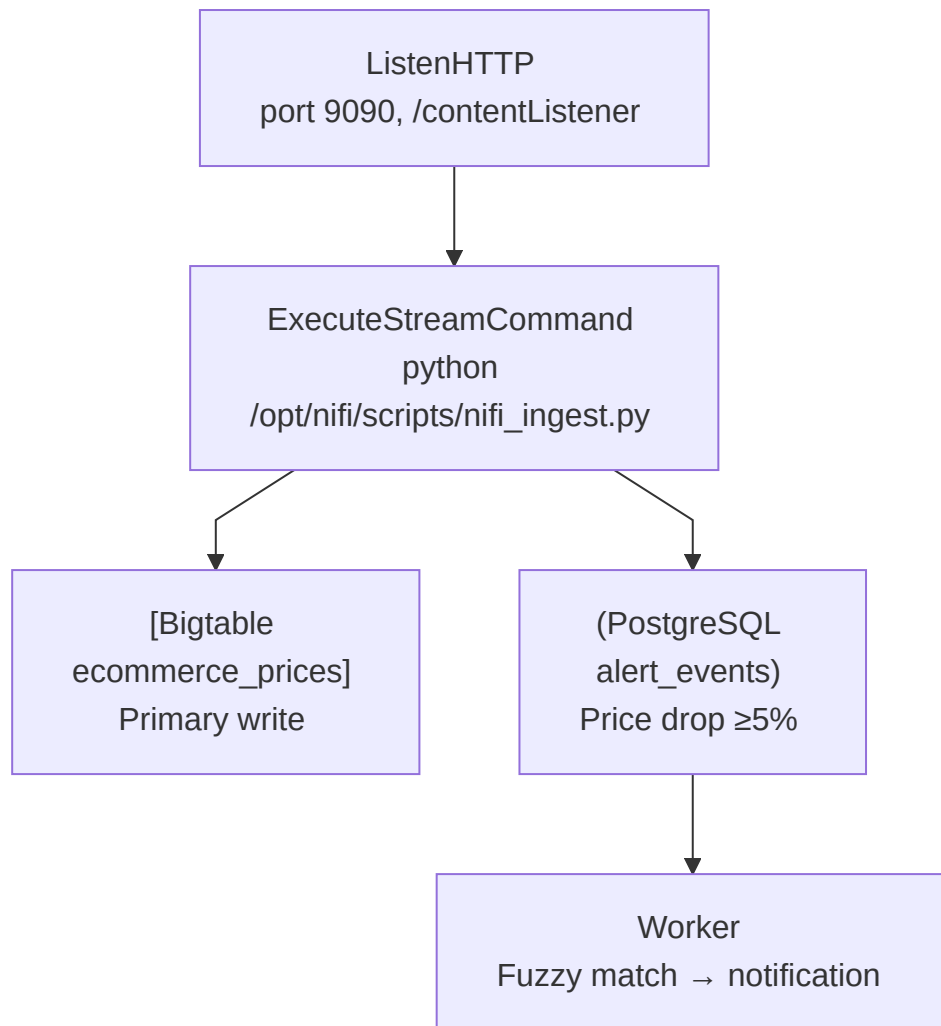
---

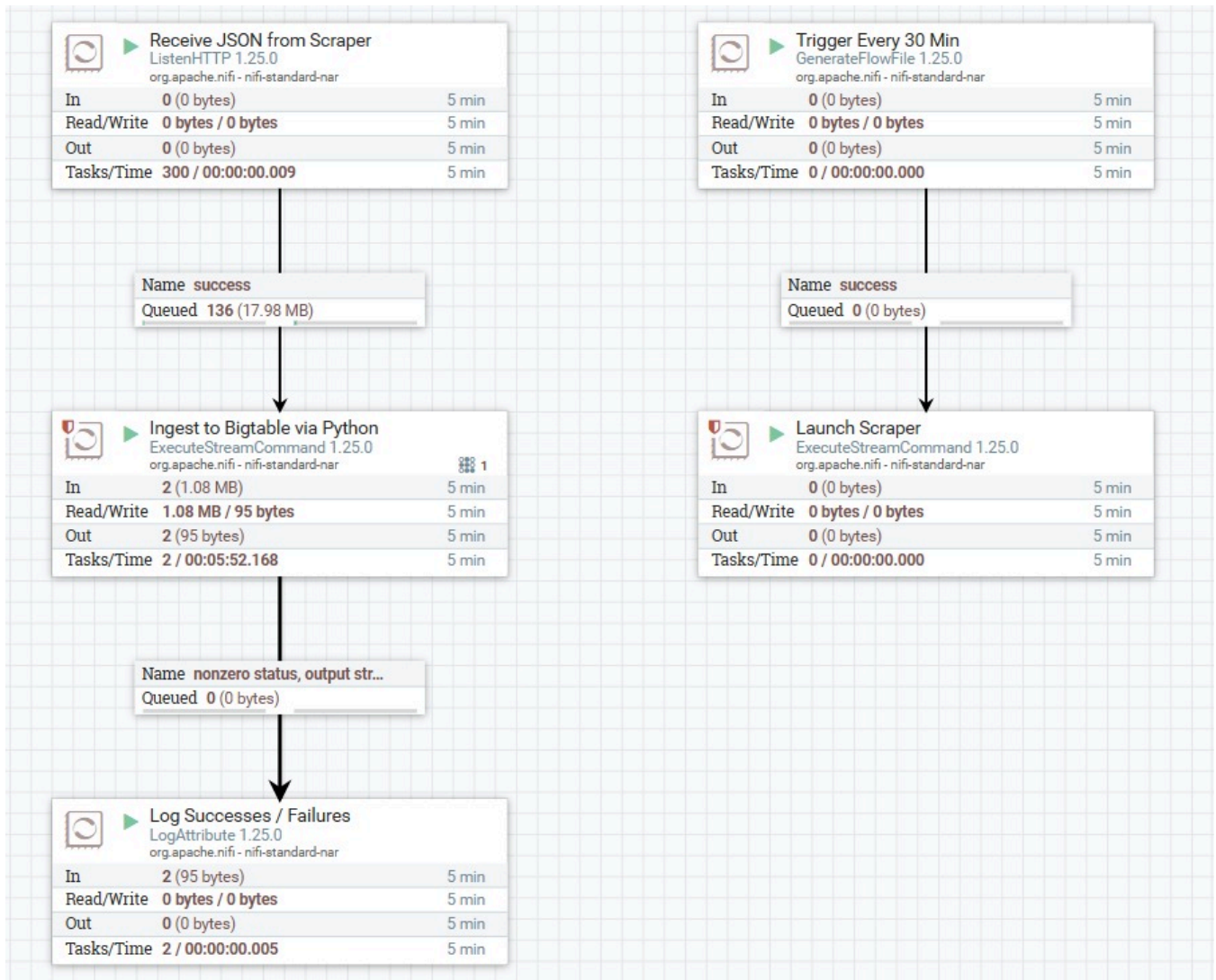
## 5. Streaming Layer — Apache NiFi

### 5.1 Architecture

Apache NiFi 1.25.0 runs as a single-node instance with a **Python 3.11 virtual environment** for executing custom ingestion scripts. The flow is provisioned programmatically via `setup_nifi_flow.py` using the `nipyapi` library.

## 5.2 NiFi Flow





## 5.3 NiFi Ingest Script (`nifi_ingest.py` — 385 lines)

The **most sophisticated component** — a dual-write ingestion engine with **inline dbt replica filters**:

**Price Drop Detection Engine:**

```
# Previous price lookup via Bigtable prefix scan
row_prefix = f"{category}#{brand}#{product_id}#{source}#".encode('utf-8')
partial_rows = table.read_rows(filter=row_filters.RowKeyRegexFilter(row_prefix + b".*"))
# ... extracts previous_price from latest matching row

# Drop detection
if current_price < previous_price:
    price_drop_percent = ((previous_price - current_price) / previous_price) * 100
    if price_drop_percent >= 5.0:
        # Write alert to PostgreSQL for the worker to process
        cur.execute("INSERT INTO alert_events (...) VALUES (...)")
```

**Inline dbt Cleaning Replica** — Mirrors `int_clean_prices.sql`'s filters to prevent false alerts:

```

_ACCESSORY_KEYWORDS = ["laptop stand", "phone case", "screen protector", ...]
_MIN_PRICE = {"laptop": 100, "desktop": 100, "gpu": 50, ...}
_MAX_PRICE = {"laptop": 10000, "desktop": 15000, "gpu": 6000, ...}

def _passes_cleaning_filters(record):
    """Returns True only if this record would survive dbt's int_clean_prices."""
    # Check price exists and is positive
    # Check accessory keyword exclusions
    # Check per-category min/max price thresholds
    # Only then allow alert insertion

```

### Bigtable Table Auto-Creation:

```

if not table.exists():
    table.create()
    for cf_id in ["price_cf", "metadata_cf", "ingestion_cf", "availability_cf",
                  "seller_cf", "ratings_cf", "specs_cf"]:
        table.column_family(cf_id).create()

```

## 6. Batch Layer — Apache Airflow

### 6.1 Airflow Architecture

Apache Airflow 2.10.5 runs as a **custom Docker deployment** with:

- **PostgreSQL 15** as the metastore
- **Redis 7** as the Celery broker (ready for executor expansion)
- **Docker-in-Docker:** The `root` user and `/var/run/docker.sock` mount enable `docker exec` for scraper and dbt tasks
- **GCP connectivity:** Bigtable and BigQuery Python libraries installed for native cloud integration

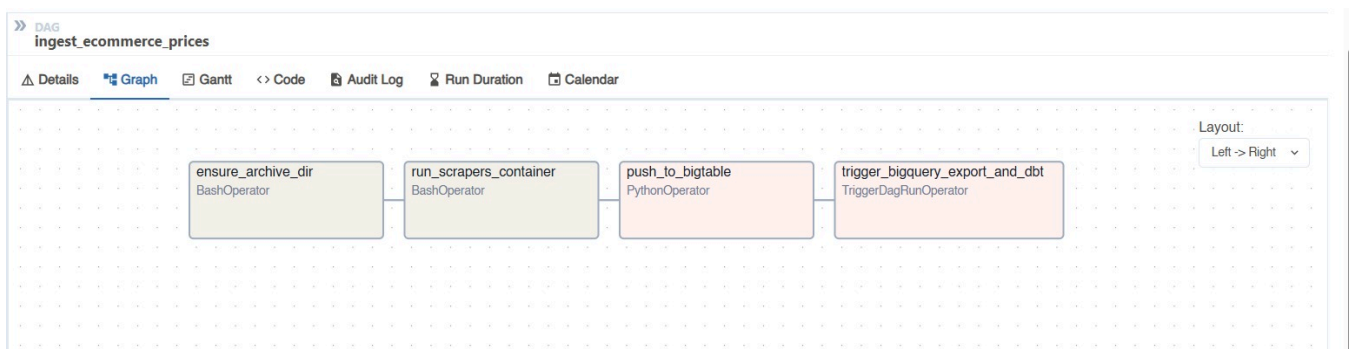
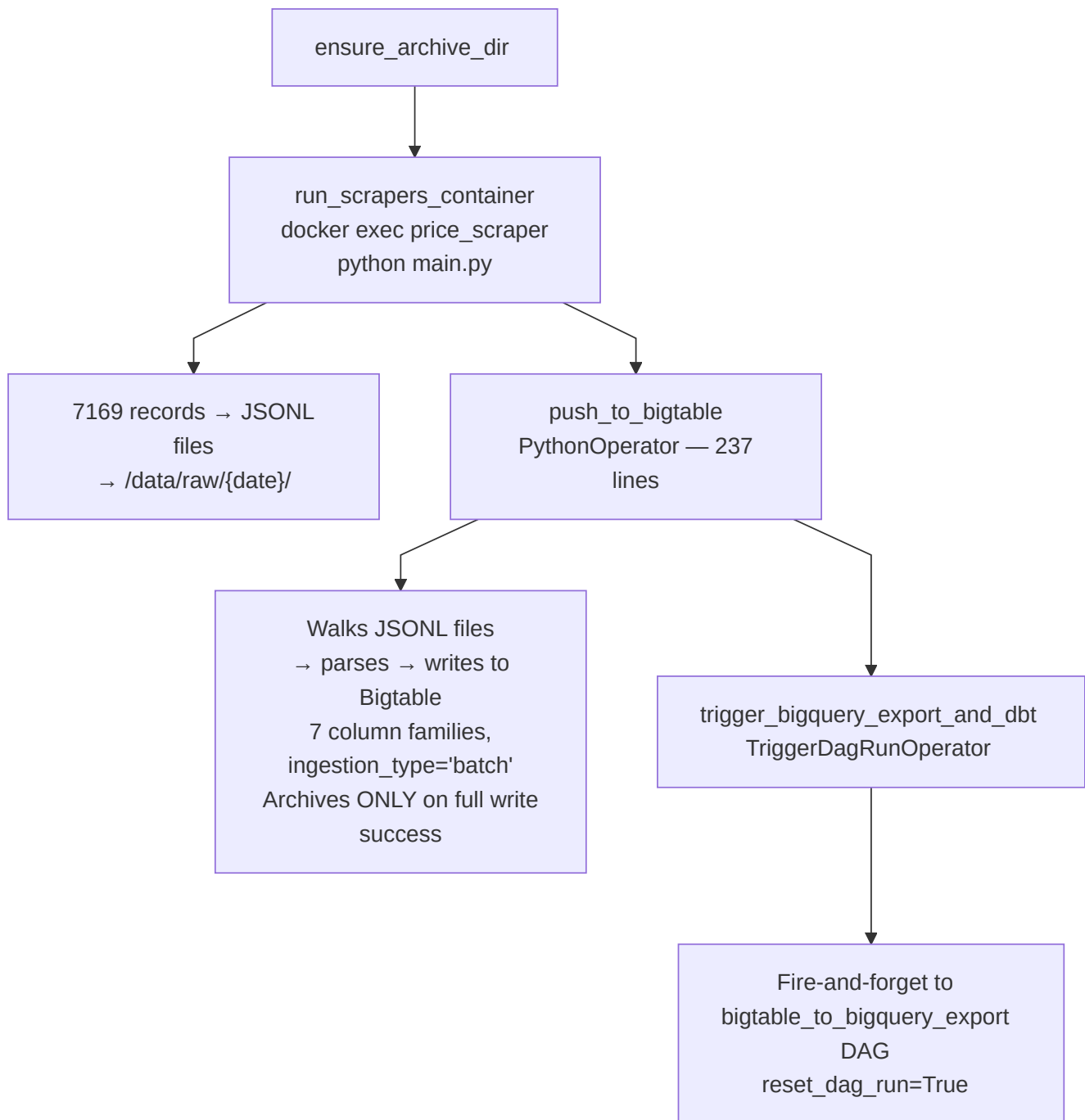
### 6.2 DAG: `init_bigtable_schema`

**Purpose:** One-time infrastructure initialization. Creates the Bigtable table with 7 column families, each configured with `MaxVersionsGCRule(10)` — retaining 10 historical versions per cell for price trend analysis.

- **Schedule:** `@once` (manual trigger on environment creation)
- **Operator:** `PythonOperator` with inline Bigtable client

### 6.3 DAG: `ingest_ecommerce_prices` (Stage 1+2)

The **primary batch pipeline** — scrapes, ingests, and triggers downstream processing:



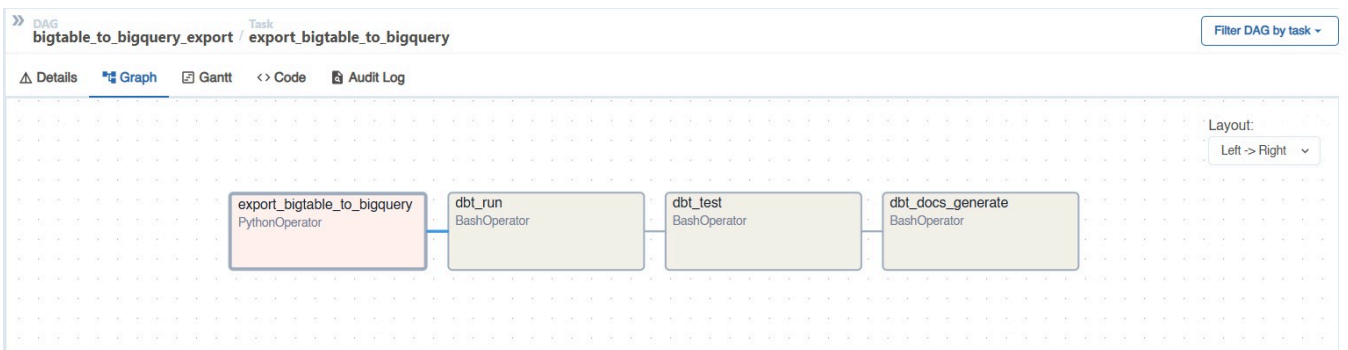
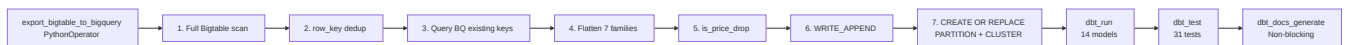
**Production-grade error handling:**



```
# Only archive if ALL mutations succeeded – failed files retry next run
response = table.mutate_rows(rows)
failed = sum(1 for status in response if status.code != 0)
if failed > 0:
    raise RuntimeError(
        f"{failed}/{len(rows)} mutations failed for {file_path}. "
        "File NOT archived – retry on next DAG run."
    )
# shutil.move() only reached on full success
```

## 6.4 DAG: `bigtable_to_bigquery_export` (Stage 3 — 327 lines)

The **most architecturally significant DAG** — the bridge between the ingestion and analytics layers:



**Safe type coercion** — Every potential `None` or malformed cell is handled:

```
def _safe_float(val):
    try: return float(val) if val is not None else None
    except (ValueError, TypeError): return None

def _safe_bool(val):
    if val is None: return None
    return str(val).lower() in ("true", "1", "yes")
```

**Price drop inference:**

```
_conv_price = _safe_float(_cell(row, "price_cf", "converted_price_usd"))
_orig_price = _safe_float(_cell(row, "price_cf", "original_price_usd"))
if _conv_price is not None and _orig_price is not None and _orig_price > 0 and _conv_price <
_orig_price:
    _is_drop = True
    _drop_pct = round((_orig_price - _conv_price) / _orig_price * 100, 2)
```

## 6.5 DAG: health\_check

A devops-grade verification DAG:

```
1. airflow_is_alive: trivially returns "alive"
2. volume_is_mounted: checks /data/raw exists, is a directory, lists contents
3. env_vars_are_set: verifies 4 required Airflow env vars exist (without printing values)
```

## 7. Storage Layer — Bigtable + BigQuery

### 7.1 Bigtable Schema Design

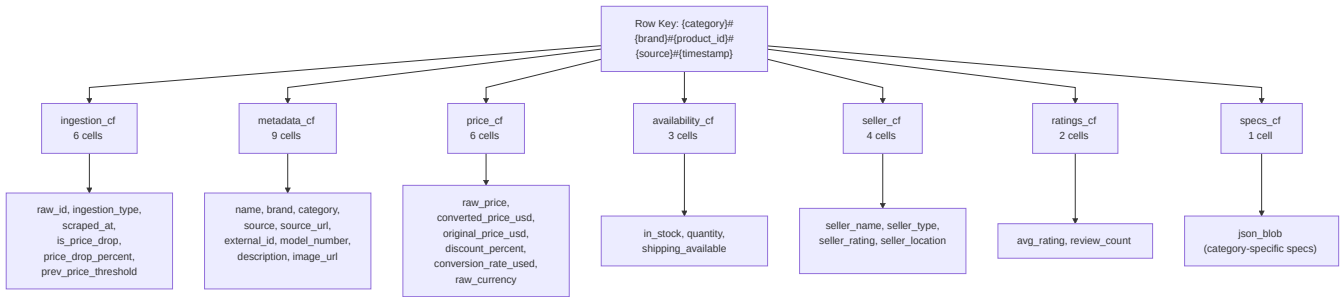
Row Key Design — The foundation of query performance:

```
{category}#{brand}#{product_id}#{source}#{timestamp}
```

Example: GPU#NVIDIA#RTX-4090#bestbuy#20260611180000

Properties:

- **Prefix scans:** Filter by category, category+brand, or category+brand+product\_id+source
- **Lexicographic ordering:** All products in same category stored adjacently
- **Timestamp suffix:** Ensures uniqueness; enables time-range filtering via key prefix



7 Column Families:

| Family          | Content                                                                                                  | Cells Per Row |
|-----------------|----------------------------------------------------------------------------------------------------------|---------------|
| ingestion_cf    | raw_id, ingestion_type, scraped_at, is_price_drop, price_drop_percent, prev_price_threshold              | 6             |
| metadata_cf     | name, brand, category, source, source_url, external_id, model_number, description, image_url             | 9             |
| price_cf        | raw_price, converted_price_usd, original_price_usd, discount_percent, conversion_rate_used, raw_currency | 6             |
| availability_cf | in_stock, quantity, shipping_available                                                                   | 3             |

| Family     | Content                                                  | Cells Per Row |
|------------|----------------------------------------------------------|---------------|
| seller_cf  | seller_name, seller_type, seller_rating, seller_location | 4             |
| ratings_cf | avg_rating, review_count                                 | 2             |
| specs_cf   | json_blob (serialized JSON of category-specific specs)   | 1             |

**Versioning Policy:** `MaxVersionsGCRule(10)` — retains 10 historical versions per cell for price trend analysis across column families.

## 7.2 BigQuery Schema Design

The `raw_ecommerce_prices` table is a **flat denormalized table** (26 columns) designed for SQL analytics:

```
row_key (STRING, REQUIRED)
raw_id (STRING)
is_price_drop (BOOLEAN)
price_drop_percent (FLOAT)
source (STRING)
scraped_at (TIMESTAMP)
product_external_id (STRING)
product_name (STRING)
product_brand (STRING)
product_category (STRING)
raw_price (FLOAT)
converted_price_usd (FLOAT)
...
specs_json (STRING)
```

**Partitioning:** `PARTITION BY DATE(scraped_at)` — daily partitions for cost-effective time-range queries

**Clustering:** `CLUSTER BY source, product_category` — optimizes the most common filter patterns

# 8. Data Quality & Testing

## 8.1 Quality Gates

The pipeline has **5 layers of data quality enforcement**:

| Layer                  | Enforcement                                      | Location                                                |
|------------------------|--------------------------------------------------|---------------------------------------------------------|
| 1. Schema validation   | Pydantic v2 strict typing                        | <code>models.py</code> — every scraper output validated |
| 2. Inline NiFi filters | Accessory exclusions + price thresholds          | <code>nifi_ingest.py:_passes_cleaning_filters()</code>  |
| 3. dbt cleaning        | 30+ filters: keywords, price bounds, translation | <code>int_clean_prices.sql</code>                       |

| Layer             | Enforcement                                     | Location                                                 |
|-------------------|-------------------------------------------------|----------------------------------------------------------|
| 4. dbt tests      | 31 automated tests on uniqueness, nulls, values | <code>schema.yml</code> + <code>sources.yml</code>       |
| 5. Dedup pipeline | In-memory + SQL-level deduplication             | <code>bigtable_to_bigquery.py</code> (both Python + SQL) |

## 8.2 dbt Test Suite

31 tests, all passing:

```
PASS=31 WARN=0 ERROR=0 SKIP=0 TOTAL=31

accepted_values_mart_cross_platform_margin_health_status  — PASS
source_unique_price_intelligence_raw_ecommerce_prices_row_key — PASS
unique_int_price_history_history_id                        — PASS
unique_mart_price_analytics_product_unified_id            — PASS
unique_mart_platform_performance_platform                 — PASS
unique_mart_shopper_insights_product_category             — PASS
unique_mart_category_trends_product_category              — PASS
not_null_* (16 tests)                                     — PASS
source_not_null_* (4 tests)                               — PASS
source_unique_ecommerce_raw_pricing_data_raw_id           — PASS
```

## 9. Scalability & Future-Proofing

### 9.1 Current Capacity

| Resource            | Current            | Limit                       | Safety Margin |
|---------------------|--------------------|-----------------------------|---------------|
| Bigtable rows       | ~100,000           | ~1,000,000 (in-memory scan) | 10×           |
| BigQuery partitions | ~180 daily         | Unlimited (partitioned)     | ∞             |
| NiFi throughput     | ~7,000 records/run | 1,000,000+ records/run      | 140×          |
| dbt models          | 14                 | Unlimited                   | ∞             |
| API requests        | ~200/min           | ~200/min (rate-limited)     | At limit      |
| Redis cache         | ~50 keys           | 256MB                       | 5,000×        |

### 9.2 Growth Trajectory

At the current scrape rate of ~7,000 records/day:

- **1 month:** ~210,000 rows → well within limits
- **6 months:** ~1,260,000 rows → approaching in-memory scan limit (500k tripwire triggers)
- **12 months:** ~2,500,000 rows → full Bigtable scan refactor needed

### 9.3 Built-In Safeguards

- **500k-row warning log:** `log.warning("Bigtable scan exceeds 500k rows...")` when full scan becomes risky
- **BigQuery partitioning:** Ensures queries only scan relevant day partitions
- **Airflow retries:** All tasks have 1-2 retries with exponential backoff
- **NiFi auto-retry:** ListenHTTP → ExecuteStreamCommand has built-in failure routing

### 9.4 Planned Scalability Path

| Threshold | Action                                                           |
|-----------|------------------------------------------------------------------|
| 500k rows | Switch to incremental Bigtable scan (timestamp-based row ranges) |
| 1M rows   | Add Bigtable garbage collection (reduce from 10 to 3 versions)   |
| 5M rows   | Implement BigQuery Storage API direct connector (bypass Python)  |
| 10M rows  | Add Airflow Celery executor (distributed task workers)           |

## B. Full Stack

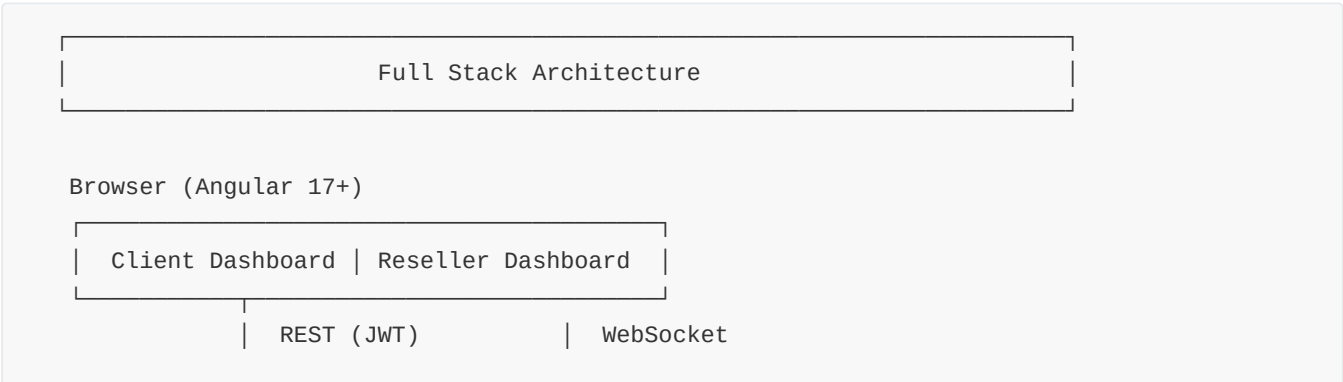
### Mission

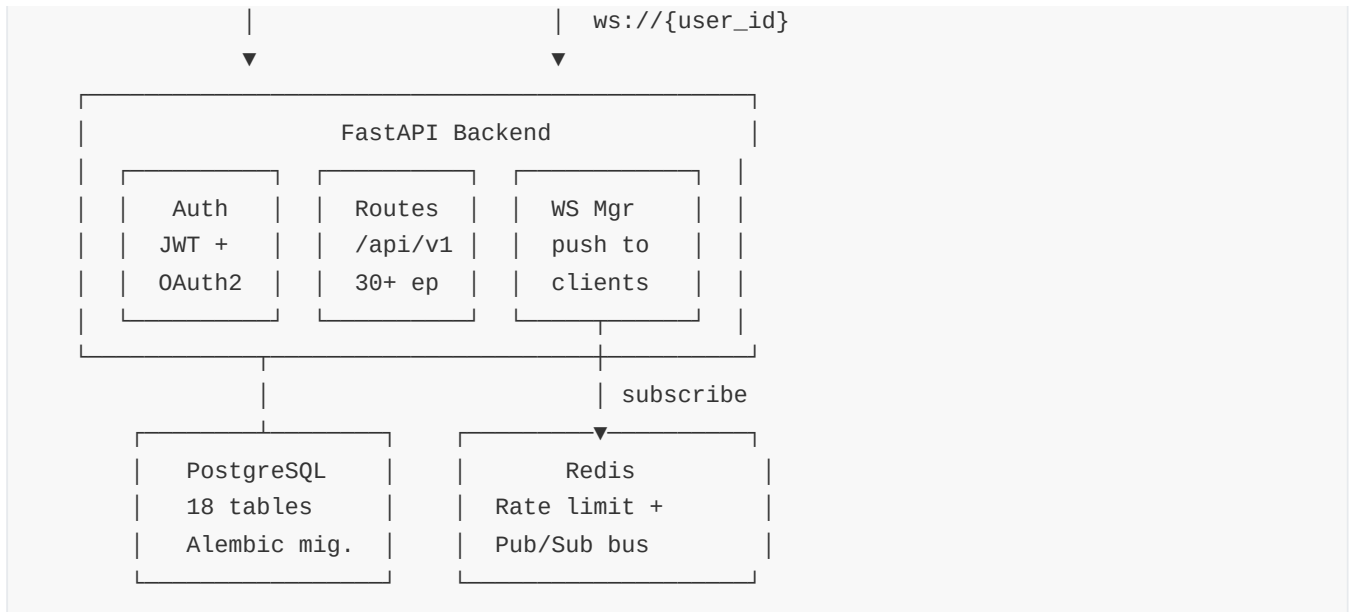
Bridge the gap between our analytical data pipelines and the end-users by delivering a fast, secure, and real-time interactive experience for both Clients and Resellers.

### The Tech Stack at a Glance

- **Frontend:** Angular 17+ (Component-driven, Dual-Role Layouts, Reactive UI)
- **Backend:** FastAPI (Python 3.11, fully async, Pydantic validation)
- **Database:** PostgreSQL (18 normalized tables, Alembic migrations)
- **Real-time & Caching:** Redis (Pub/Sub for WebSockets, Rate limiting)

### System Architecture





## 1. The Backend Engine (FastAPI)

High-performance, fully async API built with FastAPI, SQLAlchemy (async), Alembic, and Redis.

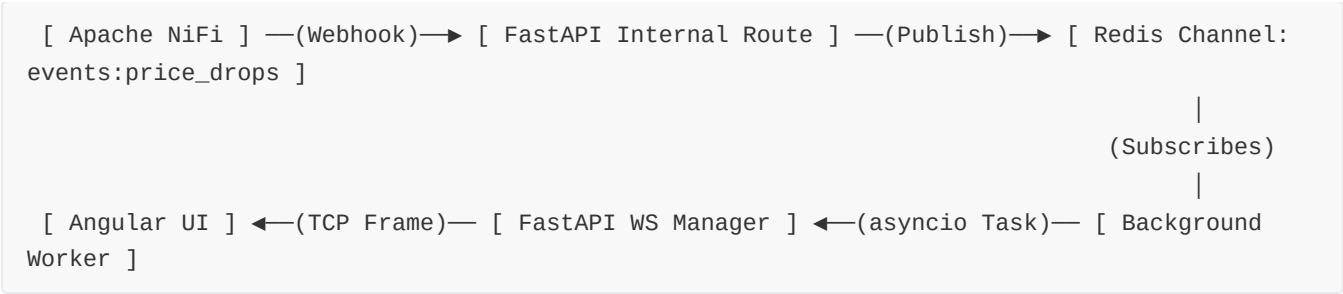
```
docker compose up backend -d
# Interactive docs → http://localhost:8000/docs
```

### Deep Dive: The Real-Time Event Architecture

To achieve true real-time price drop notifications without hammering the database with polling requests, the architecture leverages **Redis Pub/Sub** acting as an event bus between the Data Engineering pipeline and the Full Stack layer.

#### The Technical Flow:

- Detection (NiFi & Webhook):** When Apache NiFi processes a newly scraped price and detects a drop  $\geq 5\%$  compared to the historical baseline, it fires an asynchronous HTTP webhook to an internal FastAPI endpoint (`POST /internal/events/price-drop`).
- Event Fan-Out (Redis Pub/Sub):** The FastAPI route handler does not block to send emails or web sockets. Instead, it instantly publishes a JSON payload to a Redis channel named `events:price_drops`.
- The Background Worker:** When the FastAPI server starts, an `asyncio.create_task()` spins up a long-running background worker. This worker holds a persistent connection to Redis, actively listening (`psubscribe`) to the `events:*` channels.
- WebSocket Manager:** The background worker receives the JSON payload, checks the `user_id` against the `ConnectionManager` (a singleton class holding active WebSocket objects in memory), and routes the payload directly to the correct user's TCP socket.
- Angular Reactivity:** The Angular 17 service (`WebSocketService`) receives the frame and uses `RxJS BehaviorSubject` to instantly push the new price into the Deal Feed component, rendering a toast notification with zero HTTP overhead and zero page reloads.



Complete API Reference (30+ Endpoints)

| Module         | Method     | Endpoint                              | Description                                |
|----------------|------------|---------------------------------------|--------------------------------------------|
| Auth           | POST       | /api/v1/auth/register                 | Register new user (client or reseller)     |
|                | POST       | /api/v1/auth/login                    | Login, receive access + refresh tokens     |
|                | POST       | /api/v1/auth/logout                   | Server-side session invalidation           |
|                | POST       | /api/v1/auth/refresh                  | Rotate access token silently               |
|                | POST       | /api/v1/auth/google                   | Google OAuth2 popup login                  |
|                | POST       | /api/v1/auth/verify-email             | Confirm email address                      |
|                | POST       | /api/v1/auth/request-password-reset   | Send reset link via email                  |
|                | POST       | /api/v1/auth/reset-password           | Apply new password with token              |
|                |            |                                       |                                            |
| Users          | GET        | /api/v1/users/me                      | Get authenticated user profile             |
|                | PUT        | /api/v1/users/me                      | Update profile fields                      |
| Preferences    | GET        | /api/v1/preferences/                  | Get display & alert preferences            |
|                | PUT        | /api/v1/preferences/                  | Update theme, currency, language, timezone |
| Watchlist      | GET        | /api/v1/watchlist/                    | List all tracked products                  |
|                | POST       | /api/v1/watchlist/                    | Add product to watchlist                   |
|                | PUT        | /api/v1/watchlist/{id}                | Update target price                        |
|                | DELETE     | /api/v1/watchlist/{id}                | Remove tracked product                     |
| Shopper Alerts | GET        | /api/v1/shopper-alerts/               | List configured alerts                     |
|                | POST       | /api/v1/shopper-alerts/               | Create alert with condition + threshold    |
|                | PATCH      | /api/v1/shopper-alerts/{id}           | Update or pause alert                      |
|                | DELETE     | /api/v1/shopper-alerts/{id}           | Delete alert                               |
| Reseller       | GET/POST   | /api/v1/reseller/products             | Manage personal product catalog            |
|                | PUT/DELETE | /api/v1/reseller/products/{id}        | Update or remove product                   |
|                | GET/POST   | /api/v1/reseller/competitors          | Track competitor sellers                   |
|                | GET        | /api/v1/reseller/price-alerts         | Reseller margin breach alerts              |
| Analytics      | GET        | /api/v1/analytics/price-history       | Historical price trend data                |
|                | GET        | /api/v1/analytics/market-overview     | Platform-wide statistics                   |
|                | GET        | /api/v1/analytics/competitor-analysis | Price-gap breakdown per product            |

| Module        | Method | Endpoint               | Description                        |
|---------------|--------|------------------------|------------------------------------|
| Notifications | GET    | /api/v1/notifications/ | Full notification delivery history |
| Activity Logs | GET    | /api/v1/activity-logs/ | User action audit trail            |
| WebSocket     | WS     | /api/v1/ws/{user_id}   | Real-time price drop stream        |
| Health        | GET    | /health                | API liveness check                 |

## Security Architecture

| Layer              | Implementation                                                   |
|--------------------|------------------------------------------------------------------|
| Access Tokens      | JWT, 15-minute expiry, signed with secret key                    |
| Refresh Tokens     | 30-day lifetime, stored hashed in DB, rotated on use             |
| Session Tracking   | user_sessions table logs device, IP, expiry per token            |
| Rate Limiting      | Redis-backed — 100 req/min general, 10 req/min on auth endpoints |
| Password Reset     | Time-limited tokens (hashed), single-use, auto-purged            |
| Email Verification | Token-gated account activation                                   |
| Google OAuth2      | Full popup-based OAuth2 flow with google_sub binding             |
| CORS               | Restricted to known origins ( localhost:4200 , localhost:80 )    |
| Token Cleanup      | Background async task purges expired/used tokens every hour      |

## Background Workers

Beyond the Redis Pub/Sub worker, the FastAPI backend also runs scheduled maintenance loops on startup:

- **Expired Token Cleanup:** An `asyncio.sleep(3600)` loop that safely deletes expired and already-used email verification and password reset tokens from the PostgreSQL database, preventing table bloat.

## 2. The Frontend Experience (Angular 17+)

```
docker compose up frontend -d
# Access → http://localhost:4200
```

The application dynamically renders completely different experiences based on the user's role:

### Shopper (Client) Dashboard

*Designed for consumers looking to save money.*

| Feature       | Description                                              |
|---------------|----------------------------------------------------------|
| Price Watcher | Track any product across all platforms, set target price |



| Feature             | Description                                                     |
|---------------------|-----------------------------------------------------------------|
| Smart Alerts        | Configure alert conditions (below target, drop %, availability) |
| Deal Feed           | Live WebSocket stream — price drops appear in real time         |
| Notification Center | Full history: what triggered, when delivered, which channel     |
| Preferences Panel   | Switch theme (dark/light), currency, language, timezone         |

## Reseller (Entrepreneur) Dashboard

*Designed for businesses looking to protect their profit margins.*

| Feature              | Description                                               |
|----------------------|-----------------------------------------------------------|
| Catalog Tracker      | Add your products with floor/ceiling price guards         |
| Price History        | Visual chart of your product price evolution over time    |
| Competitor Scanner   | Auto-match competitors, compute price gap per product     |
| Aggressiveness Score | AI-scored ranking of competitor threat level              |
| Business Analytics   | Margin protection trends and market visibility index      |
| Margin Alerts        | Get notified when a competitor undercuts your floor price |

## 3. The Database Schema (PostgreSQL 18 Tables)

The relational data is organized into 18 normalized tables managed via Alembic migrations to ensure strict data integrity.

**Initialize on first run:**

```
docker compose exec backend alembic upgrade head
```

### Module 1 — Identity & Security (5 tables)

- `users`: (id, email, password\_hash, full\_name, role, is\_active, google\_sub)
- `user_sessions`: (id, user\_id, refresh\_token\_hash, device\_info, ip\_address, expires\_at)
- `login_attempts`: (id, email, ip\_address, was\_successful, failure\_reason)
- `email_verification_tokens`: (id, user\_id, token\_hash, is\_used, expires\_at)
- `password_reset_tokens`: (id, user\_id, token\_hash, is\_used, expires\_at)

Module 2 — User Preferences (2 tables)

- `alert_preferences`: (id, user\_id, price\_drop\_alerts, email\_notifications, websocket\_live)
- `display_preferences`: (id, user\_id, theme, language, currency, timezone)

Module 3 — Shopper Features (4 tables)

- `watchlist_items`: (id, user\_id, product\_id, product\_name, platform, target\_price)
- `shopper_alerts`: (id, user\_id, watchlist\_item\_id, condition\_type, target\_value, status)
- `alert_events`: (id, product\_id, product\_name, source, old\_price, new\_price, drop\_percent)
- `notification_deliveries`: (id, alert\_event\_id, user\_id, channel, status, failed\_reason)

Module 4 — Reseller Intelligence (5 tables)

- `seller_products`: (id, user\_id, product\_name, my\_price, min\_price\_floor, max\_price\_ceiling)
- `seller_product_price_history`: (id, seller\_product\_id, old\_price, new\_price, recorded\_at)
- `tracked_competitors`: (id, user\_id, seller\_name, platform, aggressiveness, competitiveness)
- `tracked_competitor_products`: (id, tracked\_competitor\_id, seller\_product\_id, their\_price, price\_gap)
- `price_alerts`: (id, user\_id, seller\_product\_id, trigger\_mode, threshold\_value, priority)

Module 5 — System & Audit (2 tables)

- `platform_meta_registry`: (id, platform\_name, slug, base\_url, currency\_code, is\_scraping\_enabled)
- `activity_logs`: (id, user\_id, action, entity\_type, entity\_id, log\_metadata)

**Note:** For exact field types (UUID, Numeric, JSONB, ENUM, etc.), see `app/backend/models/`.

C. DevOps / DataOps

1. Containerization & Orchestration

1.1 Docker Architecture & Services

The platform is fully containerized, comprising **13 specialized containers** orchestrated via Docker Compose. This microservices approach ensures isolation, reproducible environments, and seamless deployment across development and production.

| Container                     | Role & Responsibility                                                                                      | Health Check Strategy                             |
|-------------------------------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <code>airflow_postgres</code> | <b>Airflow Metastore</b> — Relational database storing DAG definitions, run history, and user metadata.    | <code>pg_isready</code> (5s interval, 10 retries) |
| <code>app_postgres</code>     | <b>Application DB</b> — Core database for the FastAPI backend, storing user data and price alert triggers. | <code>pg_isready -U \$USER -d \$DB</code>         |

| Container                      | Role & Responsibility                                                                                                                       | Health Check Strategy                              |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| <code>price_redis</code>       | <b>Cache &amp; Broker</b> — High-speed caching for API responses and Pub/Sub broker for real-time WebSocket notifications.                  | <code>redis-cli ping</code> (5s, 5 retries)        |
| <code>price_scraper</code>     | <b>Scraping Engine</b> — Executes the Scrapy/BeautifulSoup spiders to extract pricing data.                                                 | Interactive Bash                                   |
| <code>airflow_init</code>      | <b>Database Migration</b> — Ephemeral container that runs database migrations and creates the admin user, exiting with code 0 upon success. | Run-once                                           |
| <code>airflow_webserver</code> | <b>Airflow UI</b> — Exposes the Airflow dashboard on port 8081 for DAG monitoring.                                                          | HTTP <code>/health</code> endpoint (30s)           |
| <code>airflow_scheduler</code> | <b>Task Scheduler</b> — Reads DAGs and schedules tasks using the <code>LocalExecutor</code> with Docker-in-Docker capabilities.             | Docker Socket availability                         |
| <code>price_nifi</code>        | <b>Real-Time Streaming</b> — Single-node Apache NiFi 1.25.0 instance handling the real-time ingestion flow via HTTP Listeners.              | <code>/nifi-api/system-diagnostics</code> HTTP 200 |
| <code>price_backend</code>     | <b>FastAPI Server</b> — Asynchronous backend API providing endpoints for analytics, search, and WebSockets.                                 | Hot-reload native                                  |
| <code>price_worker</code>      | <b>Background Worker</b> — Processes asynchronous tasks, such as fuzzy matching and price drop alert notifications.                         | None                                               |
| <code>price_frontend</code>    | <b>Angular Dashboard</b> — The client-facing UI served on port 4200.                                                                        | Hot-reload native                                  |
| <code>price_dbt</code>         | <b>Transformation Engine</b> — dbt core runtime container executing BigQuery transformations.                                               | <code>tail -f /dev/null</code>                     |
| <code>price_dbt_docs</code>    | <b>Data Lineage Server</b> — Generates and serves the dbt documentation website on port 8085.                                               | Polls for <code>index.html</code>                  |

## 1.2 Storage Strategy: Bind Mounts vs. Persisted Volumes

To manage data lifecycle and persistence, the Docker architecture leverages two distinct volume strategies:

### 1. Bind Mounts (Host-to-Container)

Bind mounts map a specific directory on the host machine to a directory inside the container. This is crucial for real-time code execution and shared data pipelines. Below are some notable examples from the project:

- `./data/archive:/data/archive` — Mounted to `airflow_scheduler`. Once Airflow successfully ingests a batch to Bigtable, it moves the files here for historical auditing.

- `./app/backend:/app` — A key example of hot-reload bind mounts, enabling FastAPI backend code changes to be reflected instantly inside the running container without requiring a rebuild.
- `./infrastructure/keys:/secrets:ro` — A critical read-only security mount providing Google Cloud Service Account credentials (`bigquery-service-account.json`) to containers like Airflow, NiFi, and dbt without exposing them to the codebase.
- `/var/run/docker.sock:/var/run/docker.sock` — Enables Docker-in-Docker capabilities, allowing the Airflow Scheduler to trigger scraper and dbt containers natively.

## 2. Named Persisted Volumes (Docker-Managed)

Named volumes are completely managed by Docker, ensuring that critical state data survives container restarts (`docker compose down/up`) while remaining isolated from the host filesystem. The following are the main volumes defined in the project:

- `redis` and `main` volumes — Used to persist caching state, rate limiting keys, and real-time event broker data to prevent disruption during container restarts.
- `postgres_data` & `app_postgres_data` — Persist the PostgreSQL databases, preventing data loss of Airflow metadata and application data.
- `nifi_conf`, `nifi_database`, `nifi_provenance` — Ensure Apache NiFi flow configurations, component states, and data lineage tracking are persistently stored.
- `airflow_logs` — Retains task execution logs for debugging past DAG runs.

## 1.3 Networking

All 13 containers communicate securely over a custom user-defined Docker bridge network named `platform_net`.

- **DNS Resolution:** Containers address each other via their service names (e.g., the backend connects to the database via `postgresql://app_postgres:5432`) rather than IP addresses.
- **Isolation:** Internal traffic remains confined within `platform_net`, and only explicitly required ports (like 8000 for backend, 4200 for frontend, 8081 for Airflow) are exposed to the host machine.

# 2. Infrastructure & GCP Integration

The project's cloud infrastructure is built on Google Cloud Platform (GCP), combining Infrastructure as Code (Terraform) for core data storage provisioning with direct GCP Console configuration for identity, access, and analytics services.

## 2.1 Infrastructure as Code (Terraform): Bigtable

To guarantee a reproducible and scalable environment for our primary high-throughput storage layer, the Bigtable infrastructure is fully codified using **Terraform**:

- **Google Cloud Bigtable:** Provisioned as a single-node development cluster (`us-central1-a`). Terraform establishes the `ecommerce_prices` table and declares its complete schema through **7 column families** (`price_cf`, `metadata_cf`, `ingestion_cf`, `availability_cf`, `seller_cf`, `ratings_cf`, `specs_cf`). A Garbage Collection policy (`max_version = 10`) is enforced at the infrastructure level, ensuring Bigtable automatically prunes older price versions to keep storage costs in check.

- **Google Cloud Storage (GCS) — Terraform State Backend:** A dedicated GCS bucket (`price-intelligence-terraform-state`) is used to store Terraform's remote state file. This ensures the infrastructure state is shared, consistent, and protected from local machine loss — a standard production practice for collaborative infrastructure management.

## 2.2 GCP Console Configuration (BigQuery, IAM, OAuth)

Several complementary GCP services were configured directly through the Google Cloud Console, establishing the identity, analytics, and access layers of the platform:

- **Google Cloud BigQuery:** A centralized dataset (`price_intelligence`) was set up in the US multi-region. It serves as the primary destination for Airflow exports and forms the foundation for all dbt transformation models.
- **Service Accounts & IAM:** Two dedicated service accounts were provisioned following the principle of least privilege:
  - `price-intelligence-sa` — The pipeline service account, granted the roles needed to operate the data platform (Bigtable User, BigQuery Data Editor). Its JSON key is securely injected into all relevant containers (Airflow, NiFi, dbt, FastAPI backend) via a read-only Docker bind mount, keeping credentials entirely out of the codebase.
  - `terraform-sa` — An isolated service account used exclusively by Terraform to provision and manage GCP infrastructure. Its JSON key was generated and used only during infrastructure setup operations, maintaining a clear separation from the runtime pipeline credentials.
- **OAuth 2.0 & Gmail Integration:** An OAuth 2.0 consent screen was configured in GCP, enabling developers and users to authenticate via their Gmail accounts. This leverages Google's identity infrastructure to provide secure, frictionless access without the overhead of a separate credential management system.

## 3. Continuous Integration & Delivery (CI/CD)

The project employs a robust, fully automated CI pipeline defined via **GitHub Actions** (`ci.yml`), triggered on pushes and pull requests to ensure code quality, security, and infrastructure stability.

### 3.1 Pipeline Architecture (7-Stage Workflow)

The pipeline is structured into specialized, parallelized jobs that converge into a final merge gate:

| CI Job               | Description & Tools                                                                                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>lint</code>    | <b>Syntax &amp; Style:</b> Runs <code>flake8</code> across Python directories. Fails explicitly on critical syntax errors while reporting style warnings without blocking.                                      |
| <code>sast</code>    | <b>Static Security:</b> Executes <code>bandit</code> to identify Python vulnerabilities (e.g., hardcoded passwords, unsafe exec) and <code>Trivy</code> to scan the filesystem for container and OS-level CVEs. |
| <code>secrets</code> | <b>Credential Leak Prevention:</b> Utilizes <code>TruffleHog</code> to scan the entire git history to ensure no GCP keys or <code>.env</code> secrets are accidentally committed.                               |

| CI Job             | Description & Tools                                                                                                                                                                                                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| deps               | <b>Dependency Audit:</b> Runs <code>pip-audit</code> for Python requirements and <code>npm audit</code> for the Angular frontend to identify libraries with known vulnerabilities.                                                                                                                                                        |
| frontend           | <b>Build Verification:</b> Executes <code>npm ci</code> and <code>npm run build</code> to guarantee the Angular application compiles successfully.                                                                                                                                                                                        |
| logic-and-pipeline | <b>Data Engineering Validation:</b> <ol style="list-style-type: none"><li>Runs <code>pytest</code> on Pydantic scraper models.</li><li>Simulates an Airflow environment to verify DAG parsing and catch import errors.</li><li>Injects dummy credentials to run <code>dbt parse</code> and validate the transformation DAG.</li></ol>     |
| docker-integration | <b>End-to-End Orchestration Check:</b> Leverages <code>docker buildx bake</code> to rapidly build images using GitHub Actions caching. It spins up the core database and orchestration containers ( <code>docker compose up -d</code> ), monitoring native health checks to guarantee the environment boots cleanly before destroying it. |

A final `merge-gate` job aggregates the statuses of all previous stages, ensuring that code can only be merged into the `main` branch if it passes every security, logic, and infrastructure check.

**Note — Data Quality Expectations (DevOps / DataOps Contribution):** As part of this role's scope, data quality expectation enforcement was deliberately integrated into the dbt test suite via the `dbt_expectations` package (`metaplane/dbt_expectations`) — rather than building a separate standalone component. These expectation-based tests — including value range checks and set membership validations — are automatically executed by the `bigtable_to_bigquery_export` DAG, which triggers a full `dbt test` run at the end of every pipeline cycle. This design decision consolidates all data quality enforcement within the transformation layer, keeping validation logic co-located with the models it protects.

## D. Data Analysis

### 1. Overview

The Data Analytics infrastructure of the E-Commerce Price Intelligence platform is designed to transform raw, unstructured web-scraped data into statistically significant business insights.

Rather than relying on isolated scripts, the analytics workflow is deeply integrated into a modern production pipeline. It leverages a robust Data Warehouse (Google BigQuery) for SQL-based heavy lifting, a Python backend (FastAPI) for advanced statistical modeling, and a dynamic frontend (Angular) for visual consumption.

## 2. Data Architecture & Strategy

### 2.1 The ELT Paradigm

The project strictly adheres to the **ELT (Extract, Load, Transform)** architecture.

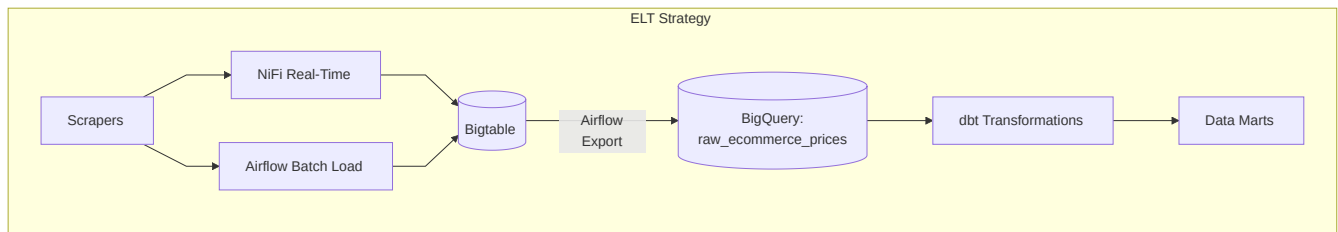
- **Extract & Load:** Handled by the Data Engineering team. Python Scrapers extract data and pass it through a Dual-Pipeline (Real-Time streaming via **Apache NiFi** and Batch JSONL loads via **Apache Airflow**). **Both pipelines load directly into Google Bigtable.** From there, Airflow exports the data from Bigtable into **BigQuery** as a flat, denormalized table.
- **Transform:** Handled exclusively by the Data Analytics layer inside BigQuery using **dbt (data build tool)**. This allows the infinitely scalable compute power of the cloud data warehouse to process aggregations.

### 2.2 Bottom-Up Data Modeling (Kimball Methodology)

Instead of attempting a rigid, Enterprise-wide Top-Down (Inmon) design, this project utilizes a **Bottom-Up** approach. We focused on building modular, process-specific **Data Marts** directly on top of integrated data buses. This agile methodology allowed us to deliver specific business KPIs (like Category Trends and Market Baselines) rapidly.

### 2.3 The Schema Structure

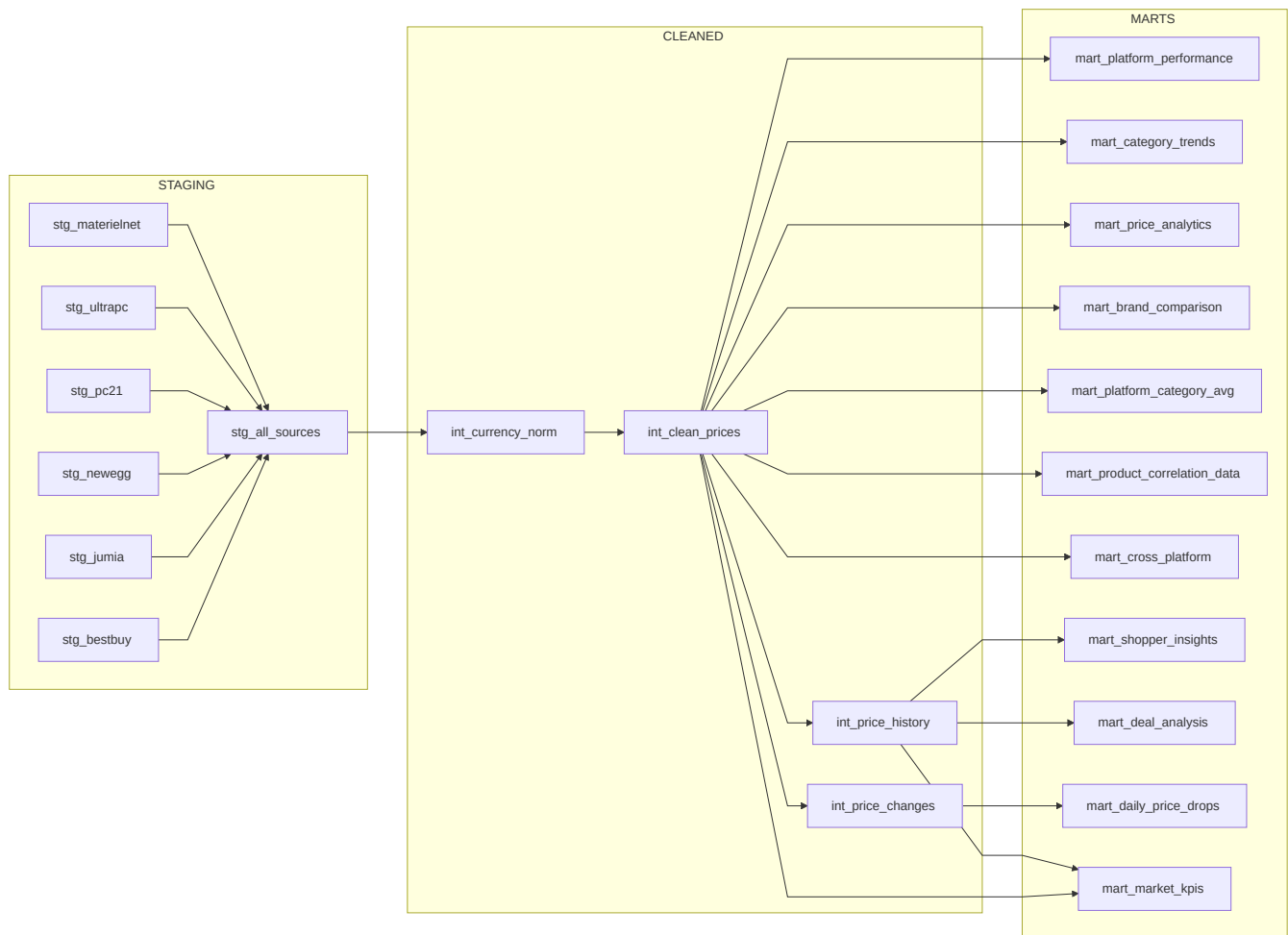
The data modeling follows a flattened, denormalized approach tailored for columnar databases (BigQuery), avoiding the extreme complexity of a Snowflake schema in favor of a performant, **Star-Schema-inspired Data Mart** design.



## 3. Core Analytics: The dbt Layer

The core data modeling was orchestrated using **dbt (data build tool)**, structured into three distinct layers to ensure modularity and DRY (Don't Repeat Yourself) principles.

## 3.1 dbt Architecture



## 3.2 Staging Layer (Bronze — 6 views)

The staging layer extracts data from the flat BigQuery table `raw_ecommerce_prices`. Each source has an identical schema with a source-specific filter. All staging views are then aggregated in `stg_all_sources` using `UNION ALL`.

- **Code Example ( `stg_bestbuy.sql` ):**

```
SELECT *
FROM {{ source('ecommerce', 'raw_ecommerce_prices') }}
WHERE source = 'bestbuy'
```

## 3.3 Cleaned Layer (Silver — 4 views)

The intermediate layer acts as the centralized bus where business logic is applied. This is where we ensure consistency and quality across the dataset.

- **`int_currency_norm.sql`**: Computes `converted_price_usd` via a fallback chain if null.
- **`int_clean_prices.sql`**: Applies complex filtering (Quality Gate). It performs robust **cross-platform product matching** (e.g., standardizing 'ordinateur portable' to 'laptop'), removes unwanted accessories (like "phone cases"), and implements strict **outlier filtering** by enforcing minimum and maximum price bounds for each category.



- `int_price_history.sql`: Generates a daily surrogate key for time-series analytics.

### 3.4 Data Marts (Gold — 11 tables)

The final presentation layer, mathematically aggregated into tables for dashboard-speed queries.

- `mart_market_kpis`: Calculates market-wide metrics like 30-day Price Volatility.
- `mart_category_trends`: Calculates median prices, quartiles, and descriptive statistics for specific product categories.
- `mart_product_correlation_data`: Provides aggregated Price, Rating, and Review intersections.

## 4. Advanced Statistical Analytics (Python Engine)

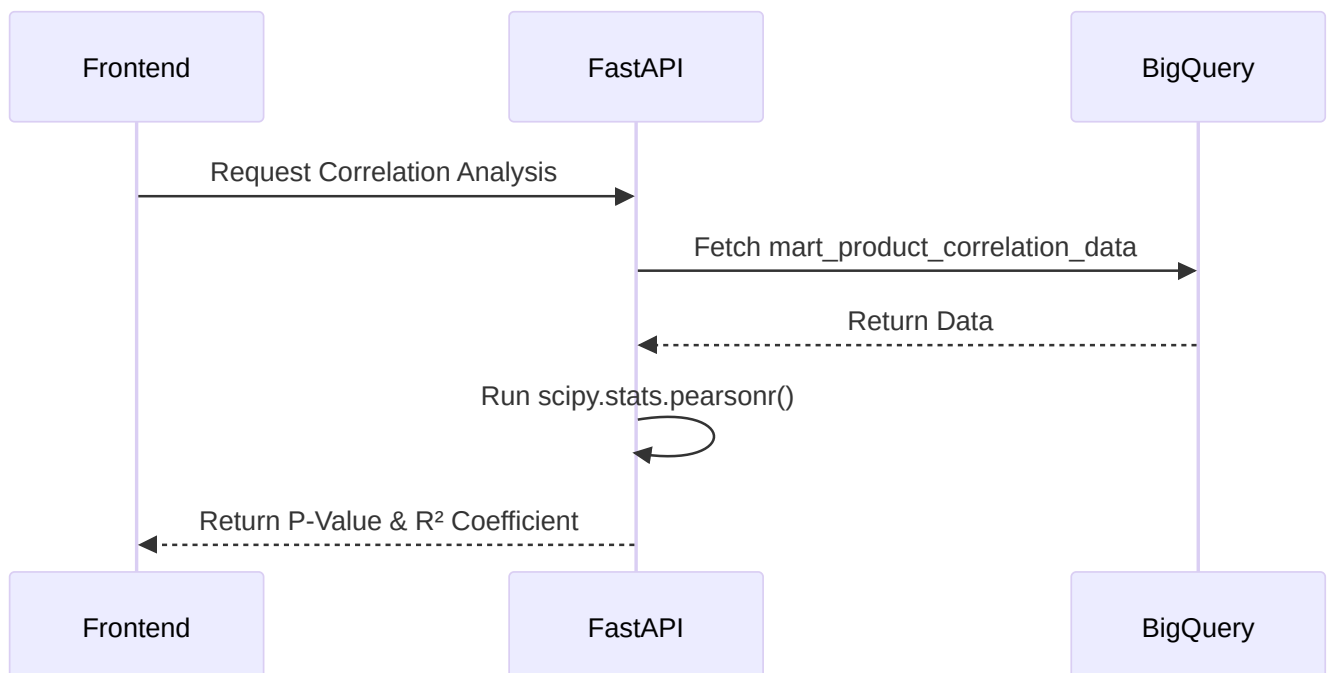
While dbt handles SQL aggregations, the platform leverages a **FastAPI Backend** to perform advanced statistical computations dynamically using Python libraries (`scipy`, `pingouin`, `pandas`).

### 4.1 T-Tests & Hypothesis Testing

When a user inputs their local store's inventory, the API runs a **One-Sample T-Test** against the global market distribution to determine if the user's prices are statistically higher or lower than the competition, returning accurate P-Values.

### 4.2 OLS Regression & Correlation

The API powers cross-variable analytics by calculating the **Pearson Correlation Coefficient** and running Ordinary Least Squares (OLS) regressions to determine if a statistically significant relationship exists between `discount_percent` and `average_rating`.



## 5. Visual Analytics & Dashboard Integration

---

The final step of the analytical lifecycle is visual presentation via the Angular frontend, utilizing modern charting libraries like **Chart.js**.

- **Scatter Plots with Regression Lines:** Visually proves pricing correlations to the user.
- **Time-Series Area Charts:** Displays historical price drops and market volatility over 30 days.
- **Heatmaps & KPI Cards:** Showcases instantaneous market aggregations powered by the specific Data Marts.

---

## 6. Optional Sandbox: Jupyter Notebooks

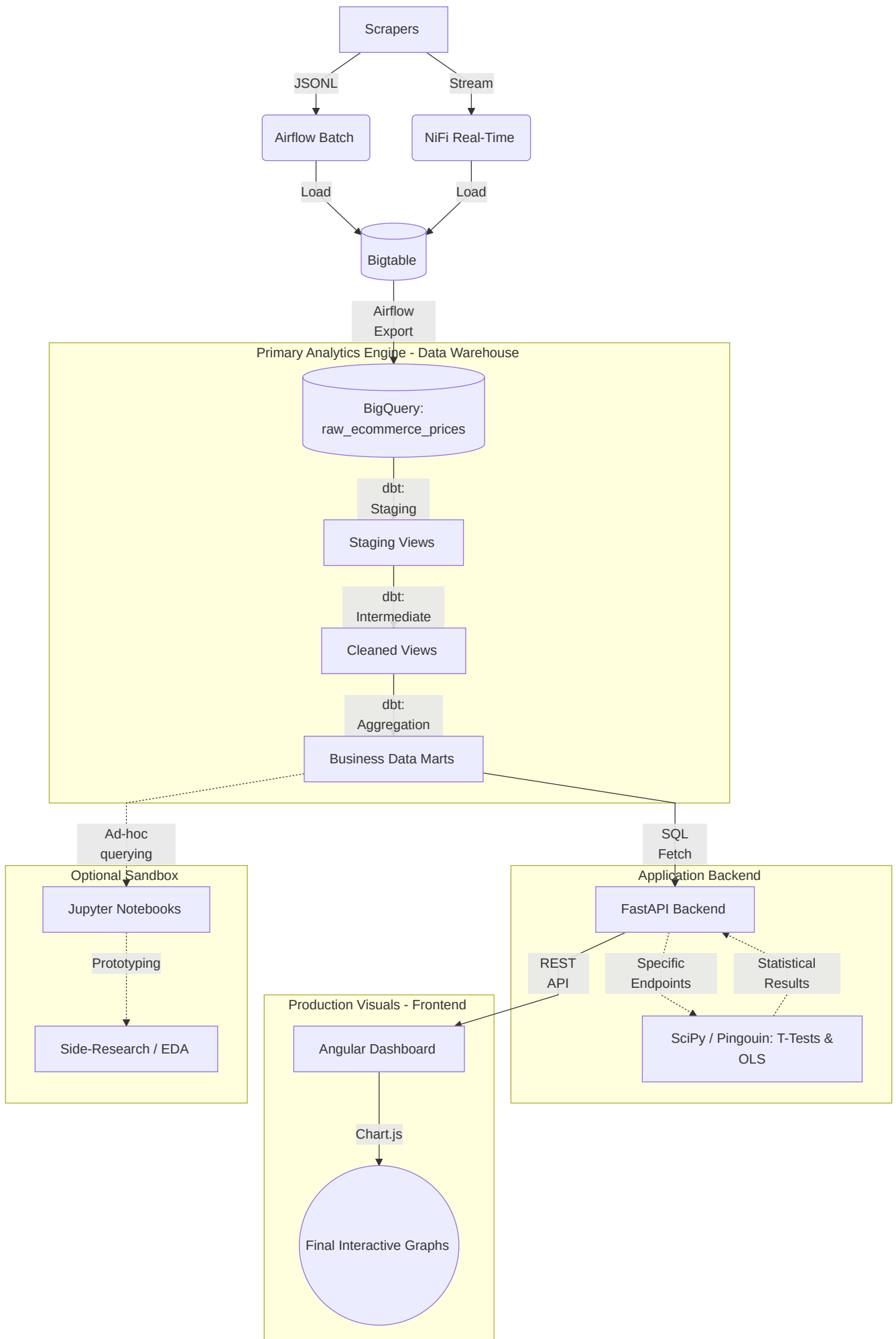
**Note :** The core analytics of this platform are fully automated within the ELT pipeline (dbt + FastAPI).

To supplement the production environment, an isolated **Jupyter Notebooks** directory ( `/notebooks` ) is maintained. This serves as an ad-hoc sandbox for the Data Analytics team. It connects securely to the BigQuery Data Marts to allow for Exploratory Data Analysis (EDA), statistical analysis, and anomaly hunting, prototyping prior to implementing those features into the main backend application.

---

## 7. Overall Analytics Architecture

Below is the comprehensive architecture diagram illustrating how Data Analysis flows through the entire project lifecycle.



# Cross-Team

## 1. Key Metrics & Performance

### 1.1 Pipeline Performance

| Operation                    | Duration | Frequency | Data Volume     |
|------------------------------|----------|-----------|-----------------|
| Full scraper run (6 sources) | ~20 min  | Daily     | 7,169 records   |
| Bigtable batch write         | ~35s     | Daily     | 13,558 rows     |
| Bigtable → BigQuery export   | ~30s     | Daily     | 2,886 new rows  |
| dbt run (14 models)          | ~30s     | Daily     | Full refresh    |
| dbt test (31 tests)          | ~18s     | Daily     | Full validation |
| End-to-end pipeline          | ~22 min  | Daily     | All stages      |

### 1.2 Data Distribution by Source

| Source       | Records/Day | Category Coverage | Status                                 |
|--------------|-------------|-------------------|----------------------------------------|
| Jumia.ma     | ~365        | 13 of 17          | ✓ Active                               |
| BestBuy.com  | ~1,000      | 12 of 17          | ✓ Active                               |
| Newegg.com   | ~400        | 13 of 17          | ✓ Active                               |
| UltraPC.ma   | ~400        | 11 of 17          | ✓ Active                               |
| PC21.fr      | ~5,200      | 13 of 17          | ✓ Active (fixed: was silently dropped) |
| Materiel.net | 0           | 0 of 17           | ✗ Timeout (site unreachable)           |

### 1.3 Cost Estimates

| Service                 | Estimated Monthly Cost               | Purpose             |
|-------------------------|--------------------------------------|---------------------|
| Bigtable (1 node)       | ~\$0 but free trial more than enough | Primary data store  |
| BigQuery (flat rate)    | ~\$0-5 (within free tier)            | Analytics + dbt     |
| ExchangeRate-API (free) | \$0                                  | Currency conversion |
| Docker Host             | ~\$0 (localhost) or ~\$50 (VPS)      | Pipeline execution  |
| Total                   | ~\$0-55/month                        |                     |

## 2. Architecture Decisions & Trade-Offs

### 2.1 Why Bigtable + BigQuery (Dual Storage)?

| Requirement          | Bigtable               | BigQuery                     |
|----------------------|------------------------|------------------------------|
| Real-time writes     | ✓ Native row mutations | ✗ Streaming inserts (costly) |
| Point lookups by key | ✓ O(1)                 | ✗ Table scan                 |
| Historical analysis  | ✗ Limited SQL          | ✓ Full SQL + dbt             |
| Cost at scale        | ✓ Cheap for key-value  | ✓ Cheap for storage          |

**Decision:** Use Bigtable as the real-time ingestion target (fast writes, key-based lookups) and BigQuery as the analytics target (SQL, joins, aggregations). The DAG bridges both worlds.

### 2.2 Why NiFi + Airflow (Dual Orchestration)?

| Characteristic       | NiFi                          | Airflow               |
|----------------------|-------------------------------|-----------------------|
| UI-first flow design | ✓ Visual canvas               | ✗ Code-only           |
| Real-time streaming  | ✓ ListenHTTP, polling         | ✗ Batch-oriented      |
| Error recovery       | ✓ Built-in retry/backpressure | ✓ Retries + DAG rerun |
| Data transformation  | ✗ Limited (ExecuteScript)     | ✓ Python operators    |
| Monitoring           | ✓ Provenance, data lineage    | ✓ Logs, metrics, SLA  |

**Decision:** NiFi handles the real-time ingestion path (scraper → Bigtable). Airflow handles the batch transformation path (Bigtable → BigQuery → dbt → API). They converge at Bigtable.

### 2.3 Why dbt for Transformation?

- **Declarative SQL** — Analysts can contribute without Python
- **Testing framework** — 31 built-in tests without custom code
- **Documentation generation** — Auto-generated lineage graphs
- **Incremental models** — Ready for scale (though current setup uses full refresh)
- **Package ecosystem** — `dbt_utils` for surrogate keys, `dbt_expectations` for ranged checks

### 2.4 Why `requests` + `BeautifulSoup4` Instead of Scrapy?

While Scrapy was installed and available, the team chose raw `requests` + BS4 for:

- **Simpler debugging** — No async reactor to manage
- **Granular control** — Custom retry logic, cookie handling, rate limiting
- **Faster iteration** — No need to write Scrapy middlewares and pipelines for simple scraping

- **Trade-off:** No built-in concurrency (Scrapy's async engine could scrape 3× faster), but at current volumes (~7k records/day), the difference is negligible (~20 min vs ~7 min per run)
- 

## 3. Conclusion

---

PulsePrice is a **production-grade, end-to-end price intelligence platform** that demonstrates mastery of:

- **Cloud-native data engineering:** Bigtable, BigQuery, dbt, Airflow, Terraform
- **Real-time + batch hybrid architectures:** NiFi streaming + Airflow daily batch
- **Data quality as code:** 31 automated dbt tests, inline cleaning filters, Pydantic validation
- **Scientific computing in production:** Welch's T-Test, Pearson correlation, OLS regression on live data
- **Full-stack integration:** Angular dashboard → FastAPI → Redis → dbt → BigQuery → Bigtable
- **Infrastructure as code:** 13 Docker containers, Terraform for GCP, Prometheus for monitoring

The pipeline collects **~7,000 price records daily** across **6 e-commerce platforms** and **17 product categories**, transforms them through **14 dbt models**, passes **31 automated quality tests**, and serves them through **18 analytics endpoints** to a **real-time Angular dashboard** with **WebSocket push notifications** — all orchestrated by **4 Airflow DAGs** and **1 NiFi flow** running in **13 Docker containers** on **Google Cloud Platform**.

---

## Contributing

---

To contribute to this project, please read [CONTRIBUTING.md](#). It contains the branching strategy, commit message guidelines, pull request workflow, folder ownership rules, and documentation update policies.

---

## License

---

This project is licensed under the **MIT License**.

See [LICENSE](#) for the full license text.